

Skillab

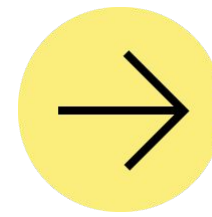
AI Agent Development

Richard Șerban Francu

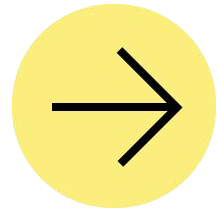
Senior AI Engineer @UCE Systems



Programa cursului



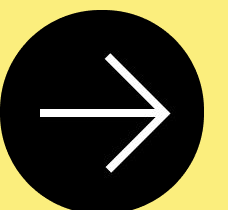
1. Noțiuni de bază — API-uri LLM și primul agent
2. Instrumente, prompturi și observabilitate
3. Procesarea și extragerea documentelor
4. Stocarea bazelor de date și RAG
5. LangGraph — Stare și fluxuri de lucru
6. Orchestrarea multi-agent
7. **Optimizarea ML — Dincolo de apelurile LLM**
8. Memorie și cache
9. Securitate și măsuri de protecție
10. Integrarea MCP
11. Inferența SageMaker
12. Training, Fine-Tuning și RLHF



Optimizarea ML — Dincolo de apelurile LLM

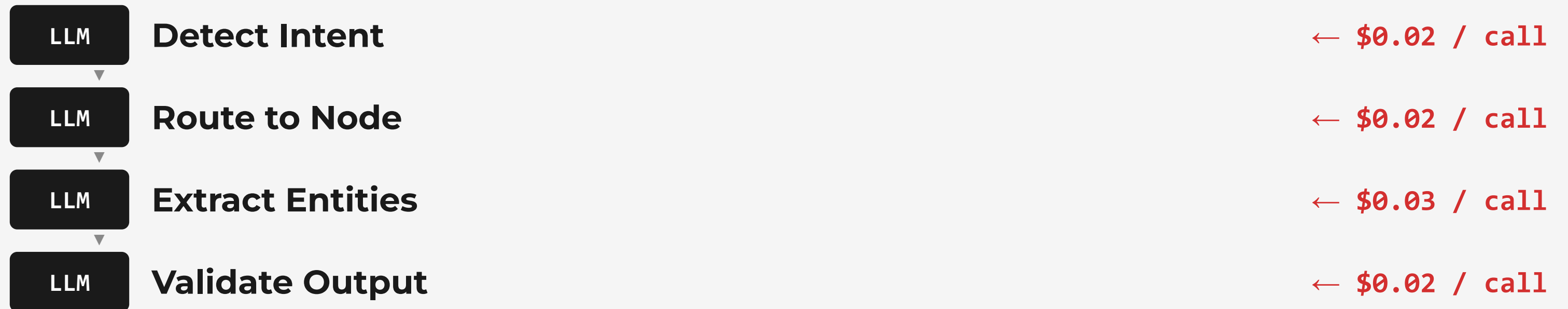
When NOT to Use LLM

Deterministic vs Probabilistic tasks
Red flags: când să NU folosești LLM
Cost comparison table



Problema: LLM Everywhere

AGENT WORKFLOW TIPIC



Response

Total: \$0.09 / request

10.000 requests/zi = **\$900/zi** = **\$27.000/lună**

Deterministic vs Probabilistic

Task	Deterministic ✓	Probabilistic ✗
Routing	If-else, regex, classifier	"Decide care agent e potrivit"
Extraction	Regex, spaCy NER	"Extrage datele din text"
Validation	JSON schema, Pydantic	"Verifică dacă e corect"
Classification	sklearn, reguli	"Ce tip de document e?"
Generation	Templates	"Scrie un răspuns"

REGULA DE AUR

Dacă output-ul e **predictibil și finit** → NU ai nevoie de LLM

Cost per 1.000 Calls

Metodă	Cost / 1.000 calls	Latență medie	Reliability
GPT-4	\$30.00	800ms	99.5%
Claude 3	\$15.00	600ms	99.5%
GPT-3.5	\$2.00	300ms	99.5%
sklearn classifier	\$0.01	5ms	100%
spaCy NER	\$0.01	10ms	100%
Regex	\$0.001	0.1ms	100%

100–1000×

ML clasic e mai ieftin decât LLM

0ms

Latență de rețea — totul rulează local

100%

Reliability — fără rate limits sau timeout-uri

Când NU folosim LLM

1 • ROUTING / NEXT ACTION

✗ **LLM:** "Analizează query-ul și decide dacă merge la search sau extract"

✓ **ML:** Classifier antrenat pe historical data

2 • ENTITY EXTRACTION

✗ **LLM:** "Extrage numele, datele și sumele din text"

✓ **ML:** spaCy NER + regex patterns

3 • INTENT DETECTION

✗ **LLM:** "Ce vrea utilizatorul să facă?"

✓ **ML:** Text classifier (sklearn / fasttext)

4 • VALIDATION

✗ **LLM:** "Verifică dacă JSON-ul e valid"

✓ **Code:** Pydantic, JSON schema



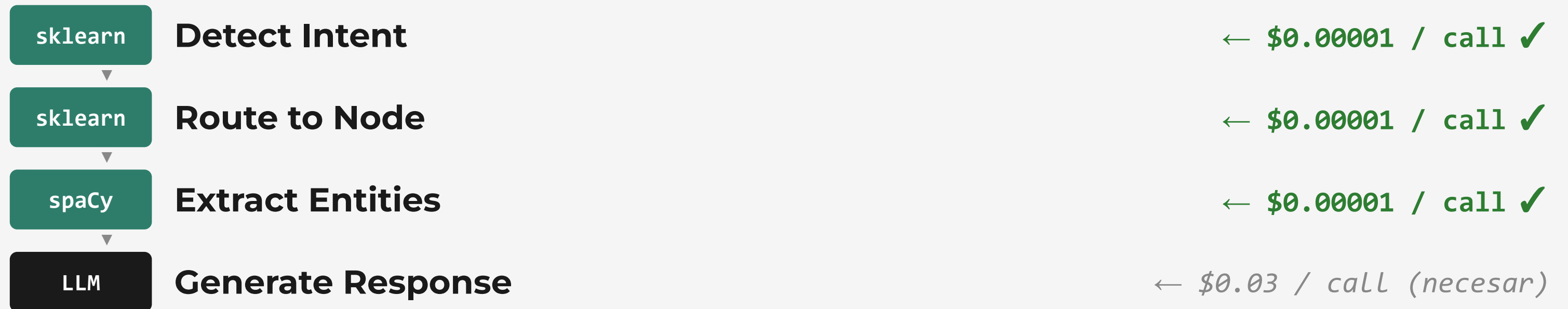
Red Flags

Semne că nu ai nevoie de LLM

- ☐ **Output-ul e unul din N opțiuni cunoscute**
→ Folosește classifier
- ☐ **Extragi date cu format fix (date, sume, email)**
→ Folosește regex sau NER
- ☐ **Faci același task de mii de ori pe zi**
→ Antrenează un model mic
- ☐ **Poți scrie reguli clare pentru decizie**
→ Folosește if-else sau rule engine
- ☐ **Ai nevoie de latență sub 50ms**
→ LLM nu e o opțiune

Agent Hybrid — soluția optimă

AGENT WORKFLOW OPTIMIZAT



Response

Total: ~\$0.03 / request

Reducere: \$0.09 → \$0.03 = **66% cost saved**

Ce am învățat

01

LLM nu e soluția universală

E scump, lent, probabilistic. Nu îl folosi pentru task-uri unde un classifier sau o regulă funcționează la fel de bine.

02

Deterministic vs Probabilistic

Output finit și predictibil → **ML clasic / reguli**

Output creativ sau complex → **LLM**

03

Costurile reale

LLM: **\$15–30** per 1.000 calls

ML clasic: **\$0.01** per 1.000 calls

Diferență de **1000×**

04

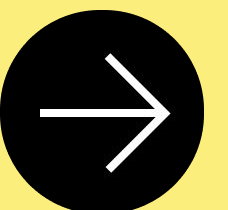
Hybrid Approach

- ML pentru routing, intent, extraction
- LLM doar pentru generare text

→ **Next:** Secțiunea 2 — Intent Detection cu scikit-learn

Intent Detection cu scikit-learn

Training data preparation
TF-IDF + LogisticRegression pipeline
Integrare în LangGraph router



Problema: Intent Detection cu LLM

```
1  # Cum facem acum în agent (SCUMP)
2  async def detect_intent_llm(query: str, llm: LLMProvider) -> str:
3      prompt = f"""Clasifică intent-ul query-ului în una din categorii:
4      - search: caută informații
5      - extract: extrage date din documente
6      - analyze: analizează și compară
7      - report: generează raport
8
9      Query: {query}
10     Răspunde doar cu categoria."""
11
12     response = await llm.generate([Message(role="user", content=prompt)])
13     return response.strip().lower()
14
15 # Cost: ~$0.02/call | Latență: ~500ms
16 # La 10.000 queries/zi = $200/zi doar pentru intent detection
17
```

Cost: ~\$0.02/call | **Latență: ~500ms**

La 10.000 queries/zi = **\$200/zi** doar pentru intent detection

Soluția: Text Classifier

INTENT CLASSIFIER

"Găsește facturile din martie"



TF-IDF Vectorizer

→ transformă text în numere



LogisticRegression

→ clasifică în 4 categorii



"search" (confidence: 0.94)

Cost: ~\$0.00001/call | Latență: ~5ms

Training Data

```
1 # Date de antrenament – poți genera cu LLM o singură dată
2 training_data = [
3     # SEARCH
4     ("găsește facturile din martie", "search"),
5     ("caută contractele cu TechSoft", "search"),
6     ("arată documentele din 2024", "search"),
7
8     # EXTRACT
9     ("extrage datele din factură", "extract"),
10    ("scoate sumele din contract", "extract"),
11    ("parsează PDF-ul atașat", "extract"),
12
13    # ANALYZE
14    ("compară vânzările Q1 vs Q2", "analyze"),
15    ("analizează trendurile", "analyze"),
16
17    # REPORT
18    ("generează raport lunar", "report"),
19    ("creează overview pentru Q1", "report"),
20 ]
```



TIP: Generează **50-100 exemple per clasă** cu LLM, apoi antrenezi classifier-ul o singură dată.

Training Pipeline



```
1  from sklearn.feature_extraction.text import TfidfVectorizer
2  from sklearn.linear_model import LogisticRegression
3  from sklearn.pipeline import Pipeline
4  import joblib
5
6  # Separăm textele și label-urile
7  texts = [t[0] for t in training_data]
8  labels = [t[1] for t in training_data]
9
10 # Creăm pipeline-ul
11 classifier = Pipeline([
12     ('tfidf', TfidfVectorizer(ngram_range=(1, 2))),
13     ('clf', LogisticRegression(max_iter=1000))
14 ])
15
16 # Antrenăm
17 classifier.fit(texts, labels)
18
19 # Salvăm pentru producție
20 joblib.dump(classifier, 'intent_classifier.joblib')
21
```

Inference în Agent



```
1  import joblib
2
3  # Încărcăm o singură dată la startup
4  classifier = joblib.load('intent_classifier.joblib')
5
6
7  def detect_intent(query: str) -> tuple[str, float]:
8      """Detectează intent-ul - înlocuiește LLM call"""
9      intent = classifier.predict([query])[0]
10     confidence = max(classifier.predict_proba([query])[0])
11     return intent, confidence
12
13
14 # Utilizare în agent node
15 intent, conf = detect_intent("găsește facturile TechSoft")
16 # → ("search", 0.94)
17
18 if conf < 0.7: # Fallback la LLM dacă nu e sigur
19     intent = detect_intent_llm(query)
20
```

Comparație Side-by-Side

Metric	LLM (Claude / GPT)	sklearn
Cost / 1.000 calls	\$20.00	\$0.01
Latență	500ms	5ms
Reliability	99.5% (API)	100% (local)
Accuracy	~95%	~92%*
Cold start	0ms	50ms (load model)
Offline	✗	✓

** Cu 100+ exemple per clasă, accuracy crește la 95%+*

Trade-off: Pierzi ~3% accuracy, câștigi 100× cost și 100× latență

Pentru majoritatea agenților, trade-off-ul e net pozitiv.

Integrare în LangGraph

```
1  def route_by_intent(state: AgentState) -> str:
2      """Router node – folosește classifier în loc de LLM"""
3      query = state["query"]
4      intent, confidence = detect_intent(query)
5
6      # Fallback la LLM pentru cazuri ambigue
7      if confidence < 0.7:
8          intent = detect_intent_llm(query)
9          state["used_llm_fallback"] = True
10
11     # Routing based on intent
12     routing = {
13         "search": "retriever_node",
14         "extract": "extractor_node",
15         "analyze": "analyst_node",
16         "report": "report_node",
17     }
18     return routing.get(intent, "default_node")
19
20
21 # În graf
22 workflow.add_conditional_edges("start", route_by_intent)
23
```

Ce am învățat

01

Intent detection nu necesită LLM

E un task de clasificare cu output finit. Un classifier antrenat pe câteva sute de exemple e suficient.

02

Pipeline simplu

TF-IDF pentru text → vectori

LogisticRegression pentru clasificare

joblib pentru save/load model

03

Hybrid Approach

sklearn pentru majoritatea query-urilor

LLM fallback când confidence < 0.7

04

Rezultate

Cost: **\$20 → \$0.01** per 1.000 calls

Latență: **500ms → 5ms**

Accuracy comparabilă (**~92-95%**)

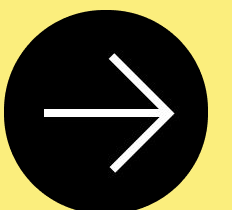
→ **Next:** Secțiunea 3 — Named Entity Recognition cu spaCy

Named Entity Recognition cu spaCy

spaCy pentru entități naturale (PER, ORG, LOC)

Regex pentru formate fixe (sume, date, ID-uri)

Pipeline combinat



Problema: Entity Extraction cu LLM

```
1  import json
2
3
4  # Cum facem acum (SCUMP)
5  async def extract_entities_llm(text: str, llm: LLMProvider) -> dict:
6      prompt = f"""Extrage entitățile din text:
7      - persoane (PER)
8      - organizații (ORG)
9      - locații (LOC)
10     - date (DATE)
11     - sume de bani (MONEY)
12
13     Text: {text}
14     Răspunde în format JSON."""
15
16     response = await llm.generate([Message(role="user", content=prompt)])
17     return json.loads(response)
18
19 # Cost: ~$0.03/call | Latență: ~600ms
20 # La 50.000 extracții/zi = $1.500/zi
21
```

Cost: ~\$0.03/call | **Latență: ~600ms**

La 50.000 extracții/zi = **\$1.500/zi**

Soluția: spaCy NER + Regex



Cost: ~\$0.00001/call | Latență: ~15ms

spaCy pentru Română

```
1 import spacy
2
3 nlp = spacy.load("ro_core_news_lg")
4
5 LABEL_MAP = {
6     "PERSON": "PER",
7     "PER": "PER",
8     "ORG": "ORG",
9     "LOC": "LOC",
10    "GPE": "LOC",
11    "DATE": "DATE",
12 }
13
14 def extract_entities_spacy(text: str) -> dict:
15     """Extrage entități cu spaCy"""
16     doc = nlp(text)
17
18     entities = {"PER": [], "ORG": [], "LOC": [], "DATE": []}
19
20     for ent in doc.ents:
21         key = LABEL_MAP.get(ent.label_)
22         if key:
23             entities[key].append(ent.text)
24
25     return entities
26
27 print(extract_entities_spacy("Ion Popescu lucreaza la Google"))
```

→ "Ion Popescu de la TechSoft București" ⇒ {"PER":["Ion Popescu"], "ORG":["TechSoft"], "LOC":["București"]}

Regex pentru Formate Fixe

```
1 import re
2 from datetime import datetime
3
4 # Patterns pentru entități cu format predictibil
5 PATTERNS = {
6     "invoice_id": r"(?:factura|fact\.\.?|#)\s*(\d{4,10})",
7     "amount": r"(\d{1,3}(?:[.]\d{3})*(?:[.]\d{2})?)\s*(RON|EUR|USD|lei)",
8     "email": r"[\w.-]+@[\w.-]+\.\w+",
9     "date_ro": r"(\d{1,2})[./-](\d{1,2})[./-](\d{2,4})",
10 }
11
12
13 def extract_with_regex(text: str) -> dict:
14     """Extrage entități cu format fix"""
15     results = {}
16     for name, pattern in PATTERNS.items():
17         matches = re.findall(pattern, text, re.IGNORECASE)
18         if matches:
19             results[name] = matches
20     return results
21
22 text = "Factura #12345 pentru 1.500,00 RON din 15.03.2024"
23 print(extract_with_regex(text))
```

→ "Factura #12345 pentru 1.500,00 RON din 15.03.2024" ⇒ {"invoice_id": ["12345"], "amount": [("1.500,00", "RON")], "date_ro": [...]}

Pipeline Combinat

```
1 def extract_all_entities(text: str) -> dict:
2     """Combină spaCy + Regex pentru extracție completă"""
3
4     # 1. spaCy pentru entități naturale
5     spacy_entities = extract_entities_spacy(text)
6
7     # 2. Regex pentru formate fixe
8     regex_entities = extract_with_regex(text)
9
10    # 3. Combinăm rezultatele
11    return {
12        "persons": spacy_entities.get("PER", []),
13        "organizations": spacy_entities.get("ORG", []),
14        "locations": spacy_entities.get("LOC", []),
15        "invoice_ids": regex_entities.get("invoice_id", []),
16        "amounts": regex_entities.get("amount", []),
17        "dates": regex_entities.get("date_ro", []),
18        "emails": regex_entities.get("email", []),
19    }
20
21
22 # În agent – înlocuiește LLM call
23 entities = extract_all_entities(document_text)
```

Comparație LLM vs spaCy + Regex

Metric	LLM	spaCy + Regex
Cost / 1.000 calls	\$30.00	\$0.01
Latență	600ms	15ms
Accuracy PER / ORG	~95%	~90%
Accuracy formate fixe	~90%	~99%
Offline	✗	✓
Customizabil	Prompt	Regex + EntityRuler

✓ **REGEX CÂȘTIGĂ**

Pe **formate fixe** (email, sume, ID-uri) — 99% accuracy, fără ambiguitate.

✓ **LLM CÂȘTIGĂ**

Pe **entități ambigue** sau context complex care necesită înțelegere semantică.

Strategie: Regex prima dată → spaCy pentru entități naturale → LLM doar dacă rămân ambiguități

Ce am **îvăţat**

01

Entity Extraction = spaCy + Regex

spaCy: persoane, organizaţii, locaţii

Regex: formate fixe (sume, date, ID-uri, email)

02

Model românesc

```
python -m spacy download ro_core_news_lg
```

Suportă PER, ORG, LOC, DATE şi MONEY pentru limba română.

03

Avantaje faţă de LLM

Cost: **\$30 → \$0.01** per 1.000 calls

Latenţă: **600ms → 15ms**

Regex: **99% accuracy** pe formate fixe

04

Când foloseşti LLM totuşi

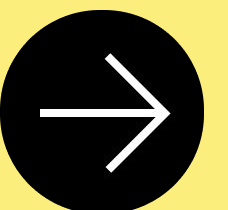
Entităţi ambigue care necesită context

Text nestructurat complex

→ **Next:** Secţiunea 4 — Embedding Optimization

Embedding Optimization

Cache pentru embeddings (evită recalculare)
sentence-transformers local (gratuit, offline)
Implementare cache simplu cu hash



Problema: Embeddings Recalculate

WORKFLOW INEFICIENT

Query: "facturile TechSoft"



[Embedding API] → 150ms + \$0.0001



Vector Search → rezultate

Query: "facturile TechSoft" (ACELAȘI!)



[Embedding API] → 150ms + \$0.0001 ← RECALCULAT!

Problema: Același text = același embedding, dar îl recalculăm de fiecare dată

Soluția: Embedding Cache

WORKFLOW CU CACHE

Query: "facturile TechSoft"



[Cache lookup] → hash(text)

MISS → [Embedding] → save → return

HIT → return cached (0ms, \$0)

Cache hit rate tipic: **40–70%** pentru agenți cu queries repetitive

Embeddings Locale

cu sentence-transformers

```
1 from sentence_transformers import SentenceTransformer
2
3 # Încarcă modelul local – o singură dată la startup
4 # Gratuit, offline, fără API
5 model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')
6
7
8 def get_embedding(text: str) -> list[float]:
9     """Generează embedding local"""
10    embedding = model.encode(text)
11    return embedding.tolist()
12
13
14 # Test
15 emb = get_embedding("facturile TechSoft din martie")
16 print(len(emb)) # 384 dimensiuni
17
```



Gratuit

Fără costuri API — model open-source



Offline

Rulează local, fără latență de rețea



~50ms

Per embedding pe CPU obișnuit

Implementare Cache Simplu

```
1 import hashlib
2 import json
3 from pathlib import Path
4
5 CACHE_FILE = Path("embedding_cache.json")
6
7
8 def load_cache() -> dict:
9     if CACHE_FILE.exists():
10         return json.loads(CACHE_FILE.read_text())
11     return {}
12
13
14 def save_cache(cache: dict):
15     CACHE_FILE.write_text(json.dumps(cache))
16
17
18 def get_embedding_cached(text: str) -> list[float]:
19     """Embedding cu cache persistent"""
20     cache = load_cache()
21     key = hashlib.md5(text.encode()).hexdigest()
22
23     if key in cache:
24         return cache[key] # Cache HIT
25
26     # Cache MISS - calculăm
27     embedding = get_embedding(text)
28     cache[key] = embedding
29     save_cache(cache)
30     return embedding
31
```

Ce am învățat

01

Problema

Același text = același embedding, dar îl recalculăm inutil de fiecare dată.

02

Soluția: hash-based cache

```
key = hashlib.md5(text.encode()).hexdigest()
```

MD5 al textului → cheie unică. Cache simplu cu dict + JSON persistent.

03

Embeddings locale

Gratuit — sentence-transformers, no API

Offline — no latency de rețea

Model: **MiniLM-L12-v2**

04

Impact

Cache hit rate: **40–70%**

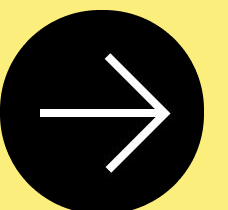
Latență: **0ms** pentru hits

Cost: **reduc proporțional** cu hit rate

→ **Next:** Secțiunea 5 — Benchmarking + Homework

Benchmarking

Cum măsurăm îmbunătățirile
Before vs After comparison
Homework assignment



Cum Măsurăm Îmbunătățirile

```
1  import time
2
3
4  def benchmark(func, *args, iterations=100):
5      """Măsoară latența medie"""
6      times = []
7      for _ in range(iterations):
8          start = time.perf_counter()
9          func(*args)
10         times.append(time.perf_counter() - start)
11
12     return {
13         "avg_ms": sum(times) / len(times) * 1000,
14         "min_ms": min(times) * 1000,
15         "max_ms": max(times) * 1000,
16     }
17
18
19 # Comparăm
20 llm_stats = benchmark(detect_intent_llm, query, iterations=20)
21 ml_stats  = benchmark(detect_intent_sklearn, query, iterations=100)
22
23 print(f"LLM: {llm_stats['avg_ms']:.0f}ms") # ~500ms
24 print(f"ML:  {ml_stats['avg_ms']:.1f}ms")  # ~5ms
25
```

Before vs After: Document Analyst

BEFORE OPTIMIZATION			
Intent Detection	[LLM]	\$0.020	500ms
Entity Extraction	[LLM]	\$0.030	600ms
Query Embedding	[API]	\$0.001	150ms
Response Gen	[LLM]	\$0.030	800ms

TOTAL		\$0.081	2.050ms

AFTER OPTIMIZATION			
Intent Detection	[sklearn]	\$0.000	5ms ✓
Entity Extraction	[spaCy]	\$0.000	15ms ✓
Query Embedding	[cached]	\$0.000	0ms* ✓
Response Gen	[LLM]	\$0.030	800ms

TOTAL		\$0.030	820ms

** 0ms pentru cache hit (~60% din cazuri)*

REDUCERE: Cost **-63%** | Latență **-60%**

La Scară: Impact Lunar

Metric (per request)	Before	After	Reducere
Cost / request	\$0.081	\$0.030	-63%
Latență	2.050ms	820ms	-60%

La 10.000 requests/zi:

	Before	After
Cost / zi	\$810	\$300
Cost / lună	\$24.300	\$9.000



Economie/lună: **\$15.300**

La 100.000 requests/zi → **\$153.000/lună economie**

Recap **Lecția 7**

Secțiune	Concept cheie	Tool
1. When NOT to use LLM	Deterministic vs Probabilistic	—
2. Classification	Intent detection fără LLM	<code>scikit-learn</code>
3. NER	Entity extraction local	<code>spaCy + regex</code>
4. Embeddings	Cache pentru a evita recalculare	<code>sentence-transformers</code>
5. Benchmarking	Măsurăm cost + latență	<code>time.perf_counter</code>

Formula de optimizare

- 1 Output predictibil și finit? → **ML clasic**
- 2 Format fix (date, sume, email)? → **Regex**
- 3 Același input frecvent? → **Cache**
- 4 Altfel... → **LLM (doar pentru generare)**

→ **Next:** Lecția 8 — Memory and Caching